

Competitive Programming

Section 17 Editorial: Topological Sorting And DAG DP

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial overview	4
2	Kahn's algorithm reminder	5
3	A. Lexicographic Course Order	6
3.1	Restatement	6
3.2	Constraints and traps	6
3.3	Observations	6
3.4	Walk-through	6
3.5	Algorithm	7
3.6	Correctness proof	7
3.7	Complexity	8
3.8	Full C++ solution	8
3.9	Full Python solution	9
3.10	Common wrong answers	10
3.11	Tests that matter	10
4	B. Longest DAG Path	11
4.1	Restatement	11
4.2	Constraints and traps	11
4.3	Observations	11
4.4	Walk-through	11
4.5	Algorithm	12
4.6	Correctness proof	12
4.7	Complexity	12
4.8	Full C++ solution	13
4.9	Full Python solution	14
4.10	Common wrong answers	14
4.11	Tests that matter	14
5	C. Project Schedule	16
5.1	Restatement	16
5.2	Constraints and traps	16
5.3	Observations	16
5.4	Walk-through	16
5.5	Algorithm	17
5.6	Correctness proof	17
5.7	Complexity	18
5.8	Full C++ solution	18
5.9	Full Python solution	19
5.10	Common wrong answers	20
5.11	Tests that matter	20
6	Stress testing	21

7	What to carry forward	22
8	D. Unique Build Order	23
8.1	Restatement	23
8.2	Observation: available choices determine uniqueness	23
8.3	Walk-through	23
8.4	Algorithm	24
8.5	Correctness proof	25
8.6	Complexity	25
8.7	Implementation notes	25
8.8	Full C++ solution	26
8.9	Full Python solution	27
8.10	Common wrong answers	28
8.11	Tests that matter	28
9	Stress testing	29
10	What to carry forward	30

1 Editorial overview

All three problems are dependency-graph problems:

Problem	Main tool
A. Lexicographic Course Order	Kahn's algorithm with min-heap
B. Longest DAG Path	Kahn's algorithm plus DP
C. Project Schedule	Kahn's algorithm plus earliest-time DP

The repeated pattern is:

```
indegree counts unmet prerequisites
indegree 0 means currently available
removing a vertex reduces indegree of its outgoing neighbors
```

2 Kahn's algorithm reminder

For a directed graph:

```
indeg[v] = number of incoming edges to v
```

Algorithm:

```
put all indeg-0 vertices into a queue or heap
while available set is not empty:
    remove one available vertex u
    append/process u
    for each edge u -> v:
        indeg[v] -= 1
        if indeg[v] == 0:
            add v to available set
```

If fewer than n vertices are processed, the graph has a directed cycle.

3 A. Lexicographic Course Order

3.1 Restatement

Find a valid order of all courses so every prerequisite comes first. If several orders exist, output the lexicographically smallest one. If a cycle makes that impossible, print **IMPOSSIBLE**.

3.2 Constraints and traps

The graph is directed. A rule **a b** means:

```
a -> b
```

Traps:

- using a normal queue gives some topological order, not necessarily the lexicographically smallest;
- cycle detection is required;
- courses are 1-indexed in input but arrays are 0-indexed in code.

3.3 Observations

At any step, the next course must have no remaining prerequisites. Among all currently available courses, choosing the smallest course number gives the lexicographically smallest order.

This is exactly Kahn's algorithm with a min-heap.

3.4 Walk-through

Dependencies:

```
1 -> 3
2 -> 3
2 -> 4
4 -> 5
```

Available courses start as:

```
1,2
```

The smallest is 1, then 2. After 2, courses 3 and 4 both become available, so choose 3 first. The final order is:

```
1 2 3 4 5
```

3.5 Algorithm

```
build adjacency list and indegree array
heap = all vertices with indegree 0
order = []

while heap not empty:
    u = pop smallest
    append u
    for v in adj[u]:
        indegree[v] -= 1
        if indegree[v] == 0:
            push v

if len(order) != n:
    print IMPOSSIBLE
else:
    print order
```

3.6 Correctness proof

Lemma 1. Every course popped from the heap has all prerequisites already in the output order.

Proof. A course enters the heap only when its indegree becomes 0. The indegree counts prerequisites not yet processed. Therefore no unprocessed prerequisite remains when the course is popped. $\square$

Lemma 2. If a valid course remains available at some step, it is in the heap.

Proof. A course is available exactly when all incoming prerequisite edges come from already processed courses. Kahn's algorithm decrements indegree for each processed predecessor, so its indegree is then 0, and the course is pushed into the heap. $\square$

Lemma 3. The algorithm's order is lexicographically smallest among all valid orders.

Proof. At the first position where two valid orders could differ, the prefix already chosen by the algorithm has been processed. The next valid course must be among the currently available courses. By Lemma 2, all such courses are in the heap, and the algorithm picks the smallest. No valid order can choose a smaller next course. $\square$

Theorem. The algorithm prints the lexicographically smallest valid order, or **IMPOSSIBLE** if none exists.

Proof. If all vertices are processed, Lemma 1 proves the order is valid, and Lemma 3 proves it is lexicographically smallest. If not all vertices are processed, the remaining vertices never reach

indegree 0, which can only happen because of a directed cycle among remaining prerequisites. No valid topological order exists. $\square$

3.7 Complexity

Time: $O((n + m) \log n)$
Memory: $O(n + m)$

3.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<vector<int>> adj(n);
    vector<int> indeg(n, 0);
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        --a;
        --b;
        adj[a].push_back(b);
        indeg[b]++;
    }

    priority_queue<int, vector<int>, greater<int>> pq;
    for (int i = 0; i < n; i++) {
        if (indeg[i] == 0) pq.push(i);
    }

    vector<int> order;
    while (!pq.empty()) {
        int u = pq.top();
        pq.pop();
        order.push_back(u);
        for (int v : adj[u]) {
            indeg[v]--;
            if (indeg[v] == 0) pq.push(v);
        }
    }
}
```

```

if ((int)order.size() != n) {
    cout << "IMPOSSIBLE\n";
} else {
    for (int i = 0; i < n; i++) {
        if (i) cout << ' ';
        cout << order[i] + 1;
    }
    cout << '\n';
}
return 0;
}

```

3.9 Full Python solution

```

import heapq
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    m = next(it)
    adj = [[] for _ in range(n)]
    indeg = [0] * n
    for _ in range(m):
        a = next(it) - 1
        b = next(it) - 1
        adj[a].append(b)
        indeg[b] += 1

    heap = [i for i in range(n) if indeg[i] == 0]
    heapq.heapify(heap)
    order = []
    while heap:
        u = heapq.heappop(heap)
        order.append(u)
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                heapq.heappush(heap, v)

    if len(order) != n:
        print("IMPOSSIBLE")
    else:
        print(" ".join(str(x + 1) for x in order))

```

```
if __name__ == "__main__":  
    main()
```

3.10 Common wrong answers

- Using a FIFO queue when lexicographic smallest order is required.
- Incrementing indegree of **a** instead of **b**.
- Forgetting to print IMPOSSIBLE when not all vertices are processed.
- Outputting zero-based course numbers.

3.11 Tests that matter

Use:

- no dependencies;
- one unique chain;
- multiple available courses at once;
- directed cycle;
- disconnected dependency graph.

4 B. Longest DAG Path

4.1 Restatement

Given a directed acyclic graph, find the largest number of edges on any directed path.

4.2 Constraints and traps

The graph is guaranteed acyclic. That guarantee is essential.

Traps:

- BFS shortest-path thinking is the wrong direction; this asks for longest path;
- longest path in a general directed graph is not solved by this DP;
- vertices with no incoming edges should start with $dp = 0$.

4.3 Observations

Define:

```
dp[v] = longest path length ending at v
```

For each edge $u \rightarrow v$, any longest path ending at u can be extended to v :

```
dp[v] = max(dp[v], dp[u] + 1)
```

Topological order guarantees that when we process u , all information needed for $dp[u]$ is already final.

4.4 Walk-through

Graph:

```
1 -> 2 -> 4 -> 5
|      |
v      v
3      5
```

Updates:

```
1->2: dp[2] = 1
1->3: dp[3] = 1
2->4: dp[4] = 2
2->5: dp[5] = 2
4->5: dp[5] = 3
```

Answer: 3.

4.5 Algorithm

```
build graph and indegrees
queue = all indegree-0 vertices
dp = zeros

while queue not empty:
    u = pop
    for v in adj[u]:
        dp[v] = max(dp[v], dp[u] + 1)
        indegree[v] -= 1
        if indegree[v] == 0:
            push v

print max(dp)
```

4.6 Correctness proof

Lemma 1. When a vertex u is popped by Kahn's algorithm in a DAG, every predecessor of u has already been processed.

Proof. u is popped only after its indegree becomes 0. Each incoming edge is removed only when its predecessor is processed. $\square$

Lemma 2. When vertex u is popped, $dp[u]$ equals the longest path length ending at u .

Proof. Every path ending at u either has length 0 if it contains only u , or ends with some edge $p \rightarrow u$. By Lemma 1, every predecessor p has already been processed, and the transition $dp[u] = \max(dp[u], dp[p] + 1)$ was applied. Therefore all possible last edges into u have been considered. $\square$

Theorem. The algorithm prints the length of the longest directed path in the DAG.

Proof. By Lemma 2, every $dp[v]$ is the longest path ending at v . Every directed path ends at some vertex, so the maximum over all vertices is the global longest path length. $\square$

4.7 Complexity

Time: $O(n + m)$
Memory: $O(n + m)$

4.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<vector<int>> adj(n);
    vector<int> indeg(n, 0);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        adj[u].push_back(v);
        indeg[v]++;
    }

    queue<int> q;
    for (int i = 0; i < n; i++) {
        if (indeg[i] == 0) q.push(i);
    }
    vector<int> dp(n, 0);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : adj[u]) {
            dp[v] = max(dp[v], dp[u] + 1);
            indeg[v]--;
            if (indeg[v] == 0) q.push(v);
        }
    }
    cout << *max_element(dp.begin(), dp.end()) << '\n';
    return 0;
}
```

4.9 Full Python solution

```
from collections import deque
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    m = next(it)
    adj = [[] for _ in range(n)]
    indeg = [0] * n
    for _ in range(m):
        u = next(it) - 1
        v = next(it) - 1
        adj[u].append(v)
        indeg[v] += 1

    q = deque(i for i in range(n) if indeg[i] == 0)
    dp = [0] * n
    while q:
        u = q.popleft()
        for v in adj[u]:
            dp[v] = max(dp[v], dp[u] + 1)
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    print(max(dp, default=0))

if __name__ == "__main__":
    main()
```

4.10 Common wrong answers

- Starting `dp` at 1 and accidentally counting vertices instead of edges.
- Processing vertices in numeric order instead of topological order.
- Trying to use shortest-path BFS.
- Forgetting isolated vertices, whose longest ending path is 0.

4.11 Tests that matter

Use:

- no edges;

- one chain;
- diamond graph with converging paths;
- disconnected DAG;
- edges from high-numbered to low-numbered vertices.

5 C. Project Schedule

5.1 Restatement

Each job has a duration. If $a \rightarrow b$, job b can start only after job a finishes. Compute earliest finish time for every job, or print IMPOSSIBLE if there is a dependency cycle.

5.2 Constraints and traps

Durations can be large and paths can contain many jobs, so C++ needs `long long`.

Traps:

- use maximum predecessor finish time, not sum of predecessor finish times;
- cycle detection is required;
- jobs with no prerequisites start at day 0;
- output finish times in original job order, not topological order.

5.3 Observations

For every job:

```
earliest_start[v] = max(finish[u] for every prerequisite u of v)
finish[v] = earliest_start[v] + duration[v]
```

Kahn's algorithm processes each prerequisite before its dependents, so we can push finish times forward along edges.

5.4 Walk-through

Durations:

```
job:      1  2  3  4  5
duration: 3  2  4  6  1
```

Dependencies:

```
1 -> 3
2 -> 3
3 -> 4
2 -> 5
5 -> 4
```

Jobs 1 and 2 start at day 0:

```
finish[1] = 3
finish[2] = 2
```

Job 3 waits for jobs 1 and 2, so it starts at $\max(3,2)=3$ and finishes at 7.

Job 5 waits for job 2, so it finishes at 3.

Job 4 waits for jobs 3 and 5, so it starts at $\max(7,3)=7$ and finishes at 13.

5.5 Algorithm

```
build graph and indegrees
earliest_start = zeros
finish = duration copy
queue = all indegree-0 jobs
processed = 0

while queue not empty:
    u = pop
    processed += 1
    finish[u] = earliest_start[u] + duration[u]
    for v in adj[u]:
        earliest_start[v] = max(earliest_start[v], finish[u])
        indegree[v] -= 1
        if indegree[v] == 0:
            push v

if processed != n:
    print IMPOSSIBLE
else:
    print finish times
```

5.6 Correctness proof

Lemma 1. When job u is processed, $\text{earliest_start}[u]$ equals the maximum finish time among all prerequisites of u .

Proof. Kahn's algorithm processes a job only after all incoming prerequisite edges have been removed, meaning every prerequisite has already been processed. Each prerequisite p applies $\text{earliest_start}[u] = \max(\text{earliest_start}[u], \text{finish}[p])$. Thus all prerequisite finish times have been considered, and no non-prerequisite can increase the value. $\square$

Lemma 2. When job u is processed, $\text{finish}[u]$ is the earliest possible finish time of u .

Proof. Job u cannot start before every prerequisite finishes, so it cannot start before the maximum prerequisite finish time. By Lemma 1, that is $\text{earliest_start}[u]$. Starting at that time is feasible because all prerequisites are done, so finishing at $\text{earliest_start}[u] + \text{duration}[u]$ is earliest possible. $\square$

Theorem. The algorithm prints correct earliest finish times, or IMPOSSIBLE if scheduling all jobs is impossible.

Proof. If all jobs are processed, Lemma 2 proves every printed finish time is correct. If not all jobs are processed, the remaining jobs are blocked by a directed cycle of prerequisites, so no schedule can complete all jobs. $\square$

5.7 Complexity

Time: $O(n + m)$
Memory: $O(n + m)$

5.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<long long> duration(n);
    for (long long &x : duration) cin >> x;
    vector<vector<int>> adj(n);
    vector<int> indeg(n, 0);
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        --a;
        --b;
        adj[a].push_back(b);
        indeg[b]++;
    }

    vector<long long> earliest_start(n, 0), finish = duration;
    queue<int> q;
    for (int i = 0; i < n; i++) {
        if (indeg[i] == 0) q.push(i);
    }

    int processed = 0;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        processed++;
    }
```

```

        finish[u] = earliest_start[u] + duration[u];
        for (int v : adj[u]) {
            earliest_start[v] = max(earliest_start[v], finish[u]);
            indeg[v]--;
            if (indeg[v] == 0) q.push(v);
        }
    }

    if (processed != n) {
        cout << "IMPOSSIBLE\n";
    } else {
        for (int i = 0; i < n; i++) {
            if (i) cout << ' ';
            cout << finish[i];
        }
        cout << '\n';
    }
    return 0;
}

```

5.9 Full Python solution

```

from collections import deque
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    m = next(it)
    duration = [next(it) for _ in range(n)]
    adj = [[] for _ in range(n)]
    indeg = [0] * n
    for _ in range(m):
        a = next(it) - 1
        b = next(it) - 1
        adj[a].append(b)
        indeg[b] += 1

    earliest_start = [0] * n
    finish = duration[:]
    q = deque(i for i in range(n) if indeg[i] == 0)
    processed = 0
    while q:
        u = q.popleft()

```

```

    processed += 1
    finish[u] = earliest_start[u] + duration[u]
    for v in adj[u]:
        earliest_start[v] = max(earliest_start[v], finish[u])
        indeg[v] -= 1
        if indeg[v] == 0:
            q.append(v)

    if processed != n:
        print("IMPOSSIBLE")
    else:
        print(" ".join(map(str, finish)))

if __name__ == "__main__":
    main()

```

5.10 Common wrong answers

- Summing all prerequisite finish times instead of taking the maximum.
- Forgetting that independent jobs can run in parallel from day 0.
- Printing in topological order instead of job-number order.
- Missing cycle detection.
- Overflowing `int` in C++.

5.11 Tests that matter

Use:

- no dependencies;
- one long chain;
- multiple prerequisites into one job;
- cycle;
- large durations.

6 Stress testing

Problem A:

```
generate small directed graphs  
compare heap Kahn output to brute-force enumeration of all permutations
```

Problem B:

```
generate DAGs using a hidden random order  
compare topological DP to brute-force DFS over all paths on small graphs
```

Problem C:

```
generate DAGs and cyclic graphs  
for DAGs, compare against repeated relaxation in topological order  
for cyclic graphs, require IMPOSSIBLE
```

The local generators cover acyclic and cyclic cases where relevant, and the checker runs both C++ and Python reference implementations.

7 What to carry forward

Topological sorting is the bridge from graph traversal to graph DP.

Carry these rules forward:

```
indegree means unmet prerequisites  
topological order means all dependencies point forward  
DAG DP is just normal DP in a dependency-respecting order  
cycle detection is order-size < n in Kahn's algorithm
```

The next graph sections build on this dependency thinking with DSU, minimum spanning trees, and weighted shortest paths.

8 D. Unique Build Order

8.1 Restatement

Each directed edge $a \rightarrow b$ says that build step a must occur before step b . There are three possible outcomes:

- a directed cycle makes every complete build order impossible;
- an acyclic graph can have two or more valid orders, so the answer is **AMBIGUOUS**;
- otherwise there is exactly one valid order, which must be printed.

The graph has up to 200000 vertices and edges. Trying permutations is useful only for a tiny brute-force tester, not for the submitted solution.

8.2 Observation: available choices determine uniqueness

Run Kahn's algorithm and consider the set of currently available vertices: those with indegree 0 after already chosen vertices have been removed.

- If it is empty before all vertices are processed, a cycle remains.
- If it ever contains two or more vertices, either one may be chosen next. Those two choices begin two distinct valid topological orders.
- If it contains exactly one vertex at every step and all vertices are processed, every position is forced. The order is unique.

The queue does not need to be a min-heap. When the order is unique there is only one possible next vertex; when it is ambiguous we print no order. A plain FIFO queue is therefore sufficient.

Do not immediately print **AMBIGUOUS** after seeing two available vertices. A separate component may still contain a cycle. Continue Kahn's algorithm and let **IMPOSSIBLE** take priority if fewer than n vertices are processed.

8.3 Walk-through

First consider a forced chain:

```
1 -> 2 -> 3 -> 4
```

The available set has one member at every stage:

```
available: {1} -> {2} -> {3} -> {4}
order:      1      2      3      4
```

So the result is:

```
UNIQUE
1 2 3 4
```

Now add two independent starting steps:

```
1 -> 3 <- 2
```

Initially both 1 and 2 have indegree 0:

```
available: {1, 2}
```

Choosing 1 first gives 1, 2, 3; choosing 2 first gives 2, 1, 3. Thus the graph is **AMBIGUOUS**.

Finally, a graph can look ambiguous locally and still be impossible overall:

```
1 -> 3 <- 2      4 <-> 5
```

Steps 1 and 2 are initially available, but 4 and 5 can never be removed. The correct answer is **IMPOSSIBLE**, not **AMBIGUOUS**.

8.4 Algorithm

```
build adjacency lists and indegree values
put every vertex with indegree 0 into a queue
ambiguous = false
order = empty list

while queue is not empty:
    if queue has at least two vertices:
        ambiguous = true

    remove one vertex u from the queue
    append u to order
    for every edge u -> v:
        decrease indegree[v]
        if indegree[v] becomes 0:
            add v to the queue

if order has fewer than n vertices:
    print IMPOSSIBLE
else if ambiguous:
    print AMBIGUOUS
else:
    print UNIQUE and order
```

8.5 Correctness proof

We prove that the algorithm prints the required classification and, when applicable, the required order.

Lemma 1. If the algorithm processes fewer than n vertices, the graph contains a directed cycle.

Proof. Kahn's algorithm stops only when no unprocessed vertex has indegree 0 within the remaining graph. Starting from any remaining vertex and repeatedly following an incoming remaining edge must eventually repeat a vertex because the remaining graph is finite. The repeated portion is a directed cycle. $\square$

Lemma 2. If at some iteration the queue contains two different vertices, the graph has at least two valid topological orders.

Proof. Both vertices have no unprocessed prerequisites, so either can be placed next after the already constructed prefix. Removing either available vertex from a DAG leaves a DAG, which Kahn's algorithm can complete. One completion starts with the first chosen vertex and another starts with the second, so the two valid orders differ at this position. $\square$

Lemma 3. If the algorithm processes all vertices and the queue has exactly one vertex at every iteration, the recorded order is the only valid topological order.

Proof. Induct on the position in the order. Before the first position, the only indegree-zero vertex is the only vertex that can legally be first. After that forced vertex is removed, the same argument applies to the unique available vertex for the next position. Every position is forced, so no different valid order exists. $\square$

Theorem. The algorithm prints the correct output.

Proof. By Lemma 1, fewer than n processed vertices means IMPOSSIBLE is correct. Otherwise the graph is acyclic. If `ambiguous` is true, Lemma 2 gives two valid orders, so `AMBIGUOUS` is correct. If it is false, Lemma 3 proves that the recorded topological order is unique, so printing `UNIQUE` and that order is correct. $\square$

8.6 Complexity

Every vertex enters the queue once, and every edge is examined once:

```
Time:     $O(n + m)$ 
Memory:  $O(n + m)$ 
```

8.7 Implementation notes

- Check `queue.size()` or `len(queue)` before removing the next vertex.
- Keep processing after finding multiple available vertices; a cycle has higher-priority output.
- Record the order only so it can be printed in the unique case.
- With $n, m \leq 200000$, use adjacency lists and an iterative queue.

8.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<vector<int>> graph(n + 1);
    vector<int> indegree(n + 1, 0);
    for (int i = 0; i < m; ++i) {
        int a, b;
        cin >> a >> b;
        graph[a].push_back(b);
        ++indegree[b];
    }

    queue<int> available;
    for (int step = 1; step <= n; ++step) {
        if (indegree[step] == 0) {
            available.push(step);
        }
    }

    bool ambiguous = false;
    vector<int> order;
    while (!available.empty()) {
        if (available.size() > 1) {
            ambiguous = true;
        }

        int u = available.front();
        available.pop();
        order.push_back(u);
        for (int v : graph[u]) {
            --indegree[v];
            if (indegree[v] == 0) {
                available.push(v);
            }
        }
    }

    if (static_cast<int>(order.size()) != n) {
        cout << "IMPOSSIBLE\\n";
    } else if (ambiguous) {
        cout << "AMBIGUOUS\\n";
    }
}
```

```

    } else {
        cout << "UNIQUE\\n";
        for (int i = 0; i < n; ++i) {
            cout << order[i] << (i + 1 == n ? '\\n' : ' ');
        }
    }
}
}

```

8.9 Full Python solution

```

import sys
from collections import deque

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m = data[:2]
    graph = [[] for _ in range(n + 1)]
    indegree = [0] * (n + 1)
    index = 2
    for _ in range(m):
        a, b = data[index], data[index + 1]
        index += 2
        graph[a].append(b)
        indegree[b] += 1

    available = deque(step for step in range(1, n + 1) if indegree[step] == 0)
    ambiguous = False
    order = []
    while available:
        if len(available) > 1:
            ambiguous = True

        u = available.popleft()
        order.append(u)
        for v in graph[u]:
            indegree[v] -= 1
            if indegree[v] == 0:
                available.append(v)

    if len(order) != n:
        print("IMPOSSIBLE")
    elif ambiguous:
        print("AMBIGUOUS")
    else:
        print("UNIQUE")
        print(*order)

```

```
if __name__ == "__main__":  
    main()
```

8.10 Common wrong answers

- Printing `AMBIGUOUS` as soon as two vertices are available, then missing a cycle in another component.
- Checking only the initial indegree-zero set. Choices can appear later.
- Using a min-heap because a previous task requested lexicographic order. This problem asks about uniqueness, not the smallest order.
- Treating a vertex with outgoing edges as unavailable. Only unmet incoming dependencies affect availability.

8.11 Tests that matter

- a one-vertex graph, which is uniquely ordered;
- a long chain, which is uniquely ordered;
- no edges on several vertices, which is ambiguous immediately;
- two independent starting vertices that later merge;
- a cycle with no initial available vertex;
- two available vertices plus a separate directed cycle, which must print `IMPOSSIBLE`.

9 Stress testing

For Problem D, keep random graphs very small and enumerate permutations of all vertices. Count valid orders only until the second one:

```
zero valid permutations -> IMPOSSIBLE  
one valid permutation  -> UNIQUE and that permutation  
two valid permutations -> AMBIGUOUS
```

This oracle is deliberately different from the submitted Kahn-based solution. It catches mistakes such as declaring ambiguity before ruling out a cycle. A separate large acyclic graph with two initial sources checks that the actual solution remains linear at 200000 vertices and edges.

10 What to carry forward

Topological sorting is more than a way to produce an order. The set of currently available vertices records what dependency choices remain:

```
no available vertex before completion -> cycle  
one available vertex at every step      -> forced order  
two available vertices at some step     -> at least two orders
```

Whenever a directed dependency graph is acyclic, process it in a topological order and ask what state can be propagated along each forward edge.